

Nama: Wildan Ahmad Gifari

NPM: 242310008

Kelas: TI 24 PA2

Matkul: Lab. Pemrograman Perangkat Bergerak

LATIHAN PERTEMUAN 8

Latihan 1: Halaman Sign-Up (Terhubung ke API)

```
import AsyncStorage from "@react-native-async-storage/async-storage";
import { useRouter } from "expo-router";
import { useState } from "react";
import {
  Alert,
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  StatusBar,
  Text,
  TouchableOpacity,
  View,
} from "react-native";
import { ADD_USER } from "../API";
import { Header } from "../components";
import { Buttons, InputText, TextPassword } from "../formUI";
import { style_auth } from "../styles";

export default function Signup() {
  const [username, setUsername] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [repassword, setRePassword] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const router = useRouter();

  const handleSignUp = async () => {
    // Validasi Username
    if (!username.trim()) {
      Alert.alert("Validation Error", "Username cannot be empty");
      return;
    }

    if (username.length < 3) {
      Alert.alert(
        "Validation Error",
        "Username must be at least 3 characters long",
      );
      return;
    }

    if (!/^[a-zA-Z0-9_]+$/.test(username)) {
      Alert.alert(
        "Validation Error",
        "Username can only contain letters, numbers, and underscores",
      );
      return;
    }

    // Validasi Email
    if (!email.trim()) {
      Alert.alert("Validation Error", "Email cannot be empty");
      return;
    }

    if (!validateEmail(email)) {
      Alert.alert("Validation Error", "Please enter a valid email address");
      return;
    }

    // Validasi Password
```

```

if (!password.trim()) {
  Alert.alert("Validation Error", "Password cannot be empty");
  return;
}

const passwordValidation = validatePassword(password);

if (!passwordValidation.isValid) {
  let errorMsg = "Password must:\n";
  if (!passwordValidation.hasMinLength)
    errorMsg += "• Be at least 6 characters long\n";
  if (!passwordValidation.hasLetter)
    errorMsg += "• Contain at least one letter\n";
  if (!passwordValidation.hasNumber)
    errorMsg += "• Contain at least one number";

  Alert.alert("Validation Error", errorMsg);
  return;
}

// Validasi Retype Password
if (!repassword.trim()) {
  Alert.alert("Validation Error", "Please retype your password");
  return;
}

if (password !== repassword) {
  Alert.alert("Validation Error", "Passwords do not match");
  return;
}

// Proses Registrasi
setIsLoading(true);

try {
  const param = { username, email, password };

  // Wajib request ke POST https://fakestoreapi.com/users
  const results = await ADD_USER(param);

  if (results.message) {
    // Request gagal / server menolak
    setIsLoading(false);
    Alert.alert("Registration Failed", "Gagal membuat akun");
    return;
  }

  if (results.data && results.data.id) {
    // Simpan data akun secara lokal (simulasi) menggunakan AsyncStorage.
    // Catatan: ini BUKAN sesi login, hanya simulasi penyimpanan data
    // pendaftaran lokal sesuai ketentuan Latihan 1. Sesi login yang
    // sebenarnya baru dibuat saat pengguna Sign In (lihat signin.jsx).
    try {
      const registeredUser = {
        id: results.data.id,
        username,
        email,
        registeredAt: new Date().toISOString(),
      };
      await AsyncStorage.setItem(
        "registeredUser",
        JSON.stringify(registeredUser),
      );
    } catch (storageError) {
      console.warn("Error saving registered user locally:", storageError);
    }

    setIsLoading(false);
    Alert.alert(
      "Registration Successful! 🎉",
      `Welcome ${username}!\n\nYour account has been created successfully.\n\nPlease sign in to continue.`,
      [
        {
          text: "Sign In Now",

```

```

        onPress: () => {
            router.replace("/signin");
        },
    ],
);
} else {
    setIsLoading(false);
    Alert.alert("Registration Failed", "Gagal membuat akun");
}
} catch (error) {
    console.error("Error during registration:", error);
    setIsLoading(false);
    Alert.alert("Registration Failed", "Gagal membuat akun");
}
};

return (
    <>
        <StatusBar style="auto" barStyle={"dark-content"} hidden={false} />
        <KeyboardAvoidingView
            behavior={Platform.OS === "ios" ? "padding" : "height"}
            style={style_auth.container}
        >
            <ScrollView
                contentContainerStyle={style_auth.scrollContainer}
                keyboardShouldPersistTaps="handled"
            >
                <View style={style_auth.content}>
                    {/* Header */}
                    <Header title={"Sign up"} />

                    {/* Form */}
                    <View style={style_auth.form}>
                        {/* Username Input */}
                        <InputText
                            data={username}
                            setData={setUsername}
                            placeholder="Username"
                        />

                        <InputText
                            data={email}
                            setData={setEmail}
                            placeholder="Email"
                            icon="mail-outline"
                            keyboardType="email-address"
                        />

                        {/* Password Input */}
                        <TextPassword password={password} setPassword={setPassword} />
                        <TextPassword
                            password={repassword}
                            setPassword={setRePassword}
                            placeholder="Retype Password"
                        />

                        {/* Sign In Button */}
                        <Buttons
                            style={[
                                style_auth.signInButton,
                                isLoading && style_auth.buttonDisabled,
                            ]}
                            onPress={handleSignUp}
                            disabled={isLoading}
                        >
                            <Text style={style_auth.signInButtonText}>
                                {isLoading ? "Processing..." : "Register"}
                            </Text>
                        </Buttons>

                        {/* Divider */}
                        <View style={style_auth.divider}>
                            <View style={style_auth.dividerLine} />
                            <Text style={style_auth.dividerText}>OR</Text>
                            <View style={style_auth.dividerLine} />
                        </View>
                    </View>
                </View>
            </ScrollView>
        </KeyboardAvoidingView>
    </>
);

```

```

        {/* Sign Up Link */}
        <View style={style_auth.signUpContainer}>
          <Text style={style_auth.signUpText}>
            Already have an account?{" "}
          </Text>
          <TouchableOpacity onPress={() => router.replace("/signin")}>
            <Text style={style_auth.signUpLink}>Sign In</Text>
          </TouchableOpacity>
        </View>
      </View>
    </View>
  </ScrollView>
</KeyboardAvoidingView>
</>
);
}

const validateEmail = (email) => {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
};

const validatePassword = (password) => {
  // Password minimal 6 karakter, harus ada huruf dan angka
  const hasMinLength = password.length >= 6;
  const hasLetter = /[a-zA-Z]/.test(password);
  const hasNumber = /[0-9]/.test(password);

  return {
    isValid: hasMinLength && hasLetter && hasNumber,
    hasMinLength,
    hasLetter,
    hasNumber,
  };
};

```

Latihan 2: Pembatasan Akses Detail Buku (Authentication Guard)

```

import { color_list } from "../../styles/StyleApps";
import Feather from "@expo/vector-icons/Feather";
import FontAwesome from "@expo/vector-icons/FontAwesome";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { BlurView } from "expo-blur";
import { LinearGradient } from "expo-linear-gradient";
import { useFocusEffect, useLocalSearchParams, useRouter } from "expo-router";
import { useCallback, useState } from "react";
import {
  ActivityIndicator,
  Alert,
  ImageBackground,
  Platform,
  ScrollView,
  StatusBar,
  StyleSheet,
  Text,
  View,
} from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";
import { ListBook } from "../../constants/list_books";
import { ButtonPill } from "../../buttons";
import Cover from "../cover";
import Header from "../header";
import Sinopsis from "../sinopsis";

export default function Detail() {
  const router = useRouter();
  const { id } = useLocalSearchParams();
  const book = ListBook.find((book) => book.id === parseInt(id));

  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [isCheckingAuth, setIsCheckingAuth] = useState(true);

```

```

// Dicek ulang setiap halaman ini fokus/dibuka (bukan cuma sekali saat
// mount), supaya kalau user Sign Out lalu mencoba kembali ke halaman ini
// lewat tombol back, guard tetap berlaku dan langsung redirect.
useFocusEffect(
  useCallback(() => {
    let isActive = true;

    const checkLoginStatus = async () => {
      try {
        const userDataString = await AsyncStorage.getItem("userData");
        const loggedIn = userDataString !== null;

        if (!isActive) return;
        setIsLoggedIn(loggedIn);
        setIsCheckingAuth(false);

        if (!loggedIn) {
          // Pengguna belum Sign In -> tidak menampilkan detail buku,
          // tampilkan pesan informatif lalu arahkan ke Sign In.
          Alert.alert(
            "Login Required",
            "Kamu harus Sign In terlebih dahulu untuk melihat detail buku.",
            [
              {
                text: "Sign In",
                onPress: () => router.replace("/signin"),
              },
            ],
            { cancelable: false },
          );
        }
      } catch (error) {
        console.error("Error checking login status:", error);
        if (!isActive) return;
        setIsLoggedIn(false);
        setIsCheckingAuth(false);
        router.replace("/signin");
      }
    };

    checkLoginStatus();

    return () => {
      isActive = false;
    };
  }, [id]),
);

const handleReadBook = async (book_id, is_free) => {
  // Guard halaman sudah memastikan pengguna login saat sampai di sini,
  // pengecekan tetap dipertahankan sebagai lapisan keamanan tambahan.
  try {
    const userDataString = await AsyncStorage.getItem("userData");

    if (!userDataString) {
      Alert.alert(
        "Login Required",
        "Please sign in to read this book",
        [
          { text: "Cancel", style: "cancel" },
          { text: "Sign In", onPress: () => router.push("/signin") },
        ],
        { cancelable: true },
      );
      return;
    }

    if (is_free) {
      router.push(`/books/read/${book_id}`);
    } else {
      router.push(`/books/subscribe/${book_id}`);
    }
  } catch (error) {
    console.error("Error in handleReadBook:", error);
    Alert.alert("Error", "Something went wrong. Please try again.");
  }
}

```

```

    }
};

// ---- Sedang memeriksa status login ----
if (isCheckingAuth) {
    return (
        <View style={styles_detail.loadingContainer}>
            <ActivityIndicator size="large" color={color_list.orange} />
        </View>
    );
}

// ---- Belum login: jangan render detail buku sama sekali ----
if (!isLoggedIn) {
    return (
        <View style={styles_detail.loadingContainer}>
            <ActivityIndicator size="large" color={color_list.orange} />
            <Text style={styles_detail.redirectText}>
                Mengarahkan ke halaman Sign In...
            </Text>
        </View>
    );
}

// ---- Sudah login: tampilkan detail buku seperti biasa ----
return (
    <ImageBackground source={book?.img} style={styles_detail.background}>
        <StatusBar style="auto" barStyle={"light-content"} />
        {Platform.OS === "ios" ? (
            <BlurView intensity={80} tint="dark" style={styles_detail.overlay} />
        ) : (
            <LinearGradient
                colors={[
                    "rgba(0,0,0,0.6)",
                    "rgba(0,0,0,0.8)",
                    "rgba(0,0,0,0.9)"
                ]}
                style={styles_detail.overlay}
            />
        )}
    <SafeAreaView style={styles_detail.container}>
        <Header book={book} />
        <ScrollView
            showsVerticalScrollIndicator={false}
            contentContainerStyle={{
                alignContent: "center",
                justifyContent: "center",
            }}
        >
            <Cover book={book} />
            <Synopsis book={book} />
        </ScrollView>
        <ButtonPress
            color={book.is_free ? color_list.white : color_list.orange}
            onAction={() => handleReadBook(book.id, book.is_free)}
        >
            <View style={styles_detail.button_container}>
                {book.is_free ? (
                    <>
                        <Feather name="book-open" size={20} color={color_list.green} />
                        <Text style={styles_detail.button_text}>Read Book</Text>
                    </>
                ) : (
                    <>
                        <FontAwesome
                            name="credit-card"
                            size={20}
                            color={color_list.white}
                        />
                        <Text
                            style={{
                                ...styles_detail.button_text,
                                color: color_list.white,
                            }}
                        >
                            Subscribe
                        </Text>
                    </>
                )}
            </View>
        </ButtonPress>
    </SafeAreaView>

```

```

        </SafeAreaView>
      </ImageBackground>
    );
  }

const styles_detail = StyleSheet.create({
  container: {
    flex: 1,
    alignContent: "center",
    justifyContent: "space-between",
    paddingHorizontal: 20,
    paddingBottom: Platform.OS === "ios" ? 15 : 25,
  },
  background: {
    flex: 1,
    resizeMode: "cover",
  },
  overlay: {
    ...StyleSheet.absoluteFillObject,
  },
  overlayAndroid: {
    ...StyleSheet.absoluteFillObject,
    backgroundColor: "rgba(0, 0, 0, 0.7)", // Dark overlay untuk Android
  },
  button_container: {
    flexDirection: "row",
    gap: 5,
    alignItems: "center",
    alignContent: "center",
  },
  button_text: {
    fontSize: 20,
    fontWeight: "bold",
    color: color_list.green,
  },
  loadingContainer: {
    flex: 1,
    backgroundColor: color_list.black || "#111",
    justifyContent: "center",
    alignItems: "center",
  },
  redirectText: {
    color: color_list.cream || "#eee",
    marginTop: 12,
    fontSize: 14,
  },
});

```

Latihan 3: Fitur Sign Out

```

// account.jsx - import yang dibutuhkan
import AsyncStorage from "@react-native-async-storage/async-storage";

// ---- Sign Out (Latihan 3) ----
// Menghapus seluruh data sesi dari AsyncStorage lalu mengarahkan
// pengguna kembali ke halaman Sign In. Setelah ini, halaman yang
// butuh login (mis. Detail Buku) tidak bisa diakses lagi.
const handleSignOut = () => {
  Alert.alert("Sign Out", "Apakah kamu yakin ingin keluar?", [
    { text: "Batal", style: "cancel" },
    {
      text: "Sign Out",
      style: "destructive",
      onPress: async () => {
        try {
          await AsyncStorage.multiRemove(["userData", "authToken"]);
        } catch (err) {
          console.warn("Error clearing session:", err);
        } finally {
          router.replace("/signin");
        }
      },
    },
  ]),
};

```

```
    });  
  };  
  
  { /* Sign Out */  
  <TouchableOpacity  
    style={{  
      flexDirection: "row",  
      alignItems: "center",  
      justifyContent: "center",  
      paddingVertical: 14,  
      marginTop: 14,  
      gap: 8,  
    }}  
    onPress={handleSignOut}  
  >  
    <Ionicons name="log-out-outline" size={20} color="#c0392b" />  
    <Text style={{ color: "#c0392b", fontWeight: "bold", fontSize: 15 }}>  
      Sign Out  
    </Text>  
  </TouchableOpacity>
```